Navegação

Rotas nomeadas

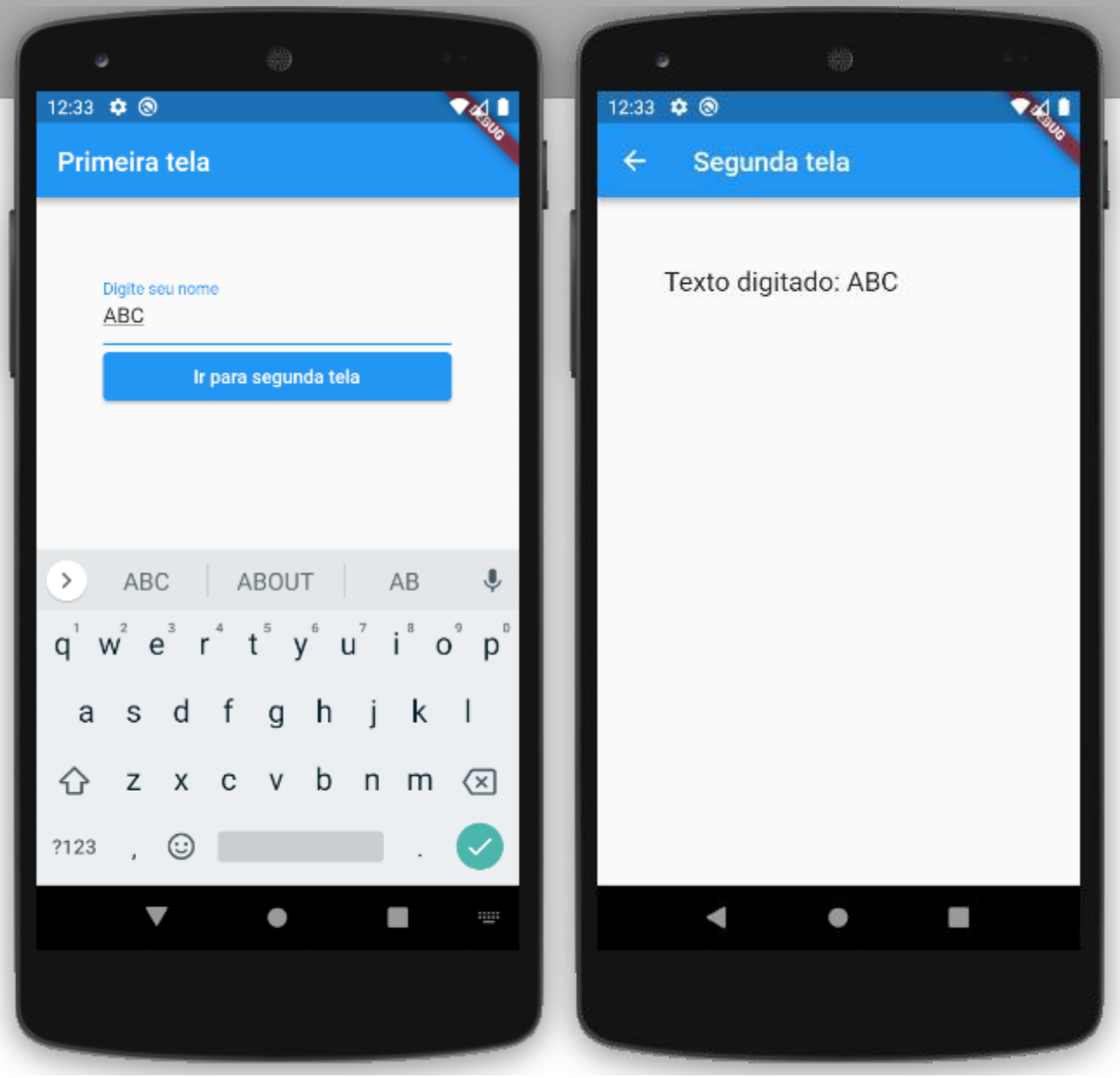
Prof. Cristiano Rodrigues

O que vamos aprender nessa aula?

- Dados entre telas
- Nomear rotas
- Layout
- List view
- AlertDialog

Passar dados entre telas

- Dando continuidade no exemplo anterior de navegação entre telas, vamos agora passar um valor de uma tela para a outra
- Observe que o nome digitado na **Primeira Tela** é mostrado na Segunda Tela



Passar dados entre telas

- Crie duas telas: *PrimeiraTela* e *SegundaTela*
- Como nossa aplicação possui apenas um *MaterialApp*, ele deverá ser incluído no método main ou em uma terceira classe.
- Altere *PrimeiraTela* e *SegundaTela* para telas semelhantes ao exemplo do slide anterior.

```
void main() {
 runApp(MaterialApp(home: PrimeiraTela()});
}

class PrimeiraTela extends StatefulWidget {
 State<PrimeiraTela> createState() =>
_PrimeiraTelaState();
}
class _PrimeiraTelaState extends State<PrimeiraTela> {
 Widget build(BuildContext context) { return …; }
}

class SegundaTela extends StatefulWidget {
 State<SegundaTela> createState() =>
_SegundaTelaState();
}
class _SegundaTelaState extends State<SegundaTela> {
 Widget build(BuildContext context) { return …; }
}
```

Passar dados entre telas

- Na *PrimeiraTela* adicione uma variável do tipo `TextEditingController` para armazenar o texto digitado. A variável será usada no textfield com o parâmetro `controller`

```
class _PrimeiraTelaState extends State<PrimeiraTela> {
 TextEditingController _nameController = TextEditingController();

 @override
 Widget build(BuildContext context) {
   …
   TextField(decoration: InputDecoration(labelText: 'Digite seu nome'),
   controller: _nameController)
   …
 }
```

Passar dados entre telas

Na primeira tela:

```
ElevatedButton(onPressed: (){
 setState(() {
   Navigator.of(context).push(
     MaterialPageRoute(builder: (context) =>
        SegundaTela(_nameController.text))
   );
 });}, child: Text('Ir para segunda tela'))
```

O texto da variável será passado da primeira para a segunda tela usando o método `push` de `Navigator`

Na segunda tela:

```
class SegundaTela extends StatefulWidget {
 String? valor;
 SegundaTela(String this.valor);
 @override
 State<SegundaTela> createState() => _SegundaTelaState();
}
```

Desta maneira, a segunda tela receberá uma String **`valor`**, como no exemplo abaixo. Como trata-se de um StatefulWidget, a variável pode ser acessada em `_SegundaTelaState` como **`widget.valor`**

Nomear rotas

- É possível nomear as rotas e usá-las para realizar a navegação entre as telas.
- Vamos usar os seguintes argumentos:
  - **initialRoute** (rota de início)
  - **routes** (outras rotas)

```
initialRoute: "/",
routes: {
  "/segunda": (context) =>
SegundaTela(),
},
```

Nomear rotas com parametros

```
MaterialApp(
  initialRoute: "/",
  onGenerateRoute: (settings) {
    if (settings.name == "/segunda") {
      final nome = settings.arguments as String;

      return MaterialPageRoute(
        builder: (context) => SegundaTela(nome: nome),
      );
    }

       return MaterialPageRoute(
      builder: (context) => PrimeiraTela(),
    );
  },
)
```

Nomear rotas

```
void main() {
 runApp(
     MaterialApp(
         home: PrimeiraTela(),
         initialRoute: "/",
         routes: {
             "/segunda": (context) => SegundaTela()
         }
     )
 );
}
```

"/" indica qual é a rota de início

"**/segunda**" é o nome dado para a rota. Podemos indicar quantas forem necessárias

Nomear rotas

PrimeiraTela:

```
ElevatedButton(onPressed: (){
 setState(() {
   Navigator.of(context).pushNamed(
    "/segunda", arguments: _nameController.text);
 });}, child: Text('Ir para segunda tela'))
```

E para navegar entre as telas, agora vamos usar o método **pushNamed()** do Navigator

E para a rota, agora usamos apenas o nome, que foi criado na função main.

SegundaTela:

```
ElevatedButton(onPressed: (){
 setState(() {
   Navigator.pop(context);
   Navigator.of(context).pushReplacementNamed("/");
 });}, child: Text('Voltar'))
```

Para voltar para a tela anterior usando rotas nomeadas, podemos adicionar um botão na segunda tela e usar a rota "/" que foi indicada na main()

Nomear rotas

SegundaTela:

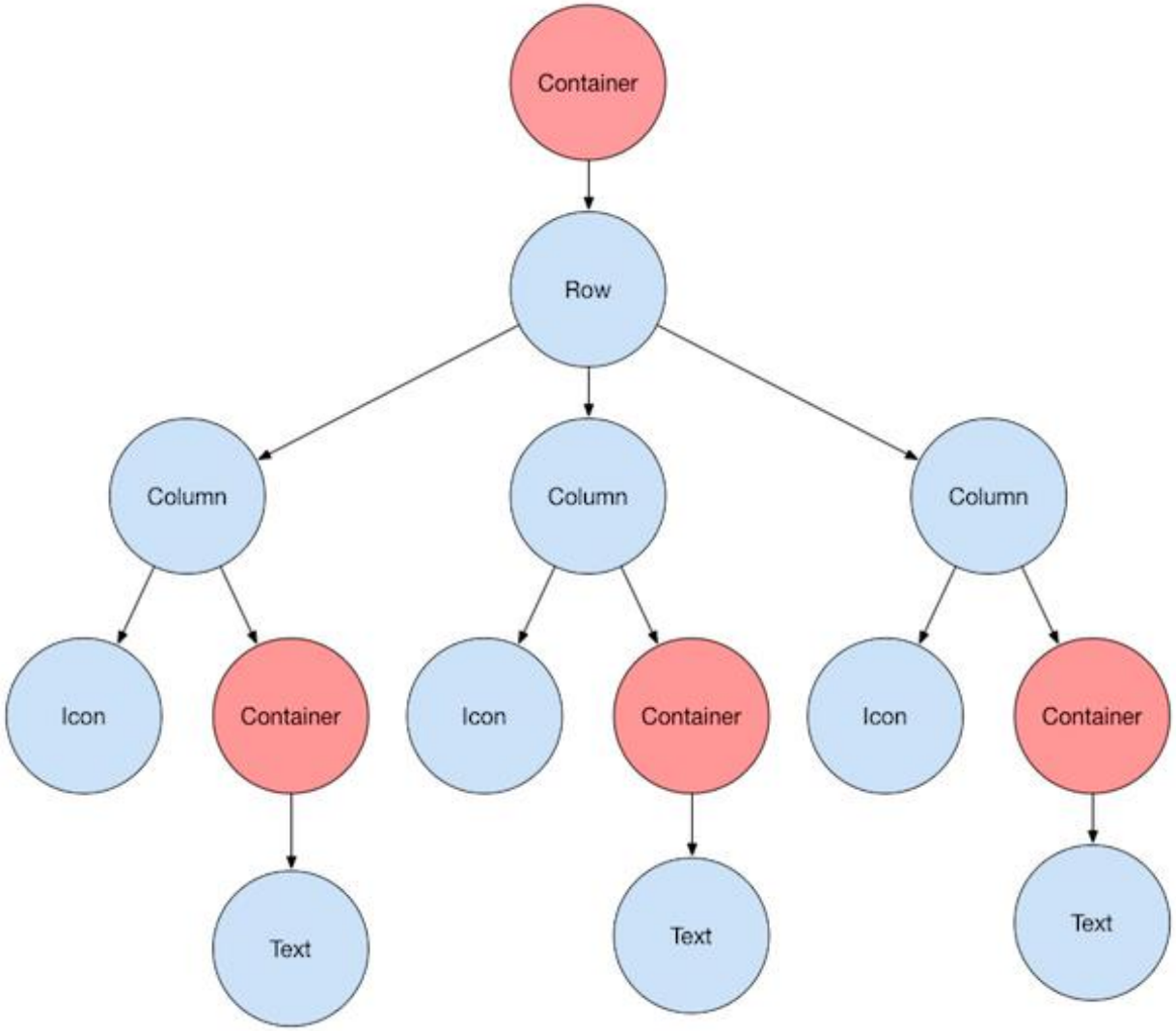
```
class _SegundaTelaState extends State<SegundaTela> {
 @override
 Widget build(BuildContext context) {
   String valor = ModalRoute.of(context)?.settings.arguments as String;
   return Scaffold(
   …
       Text('Texto digitado: ${valor}',
       style: TextStyle(fontSize: 20)),
   …
   )
```

Observe que, como a variável está sendo definida dentro da mesma classe, ela pode ser acessada diretamente por seu nome `valor`, em vez de `widget.valor`

Para receber os argumentos passados por `arguments` usando rotas nomeadas (slide anterior), podemos usar o `ModalRoute` e especificar de qual contexto e tipo será recebido.

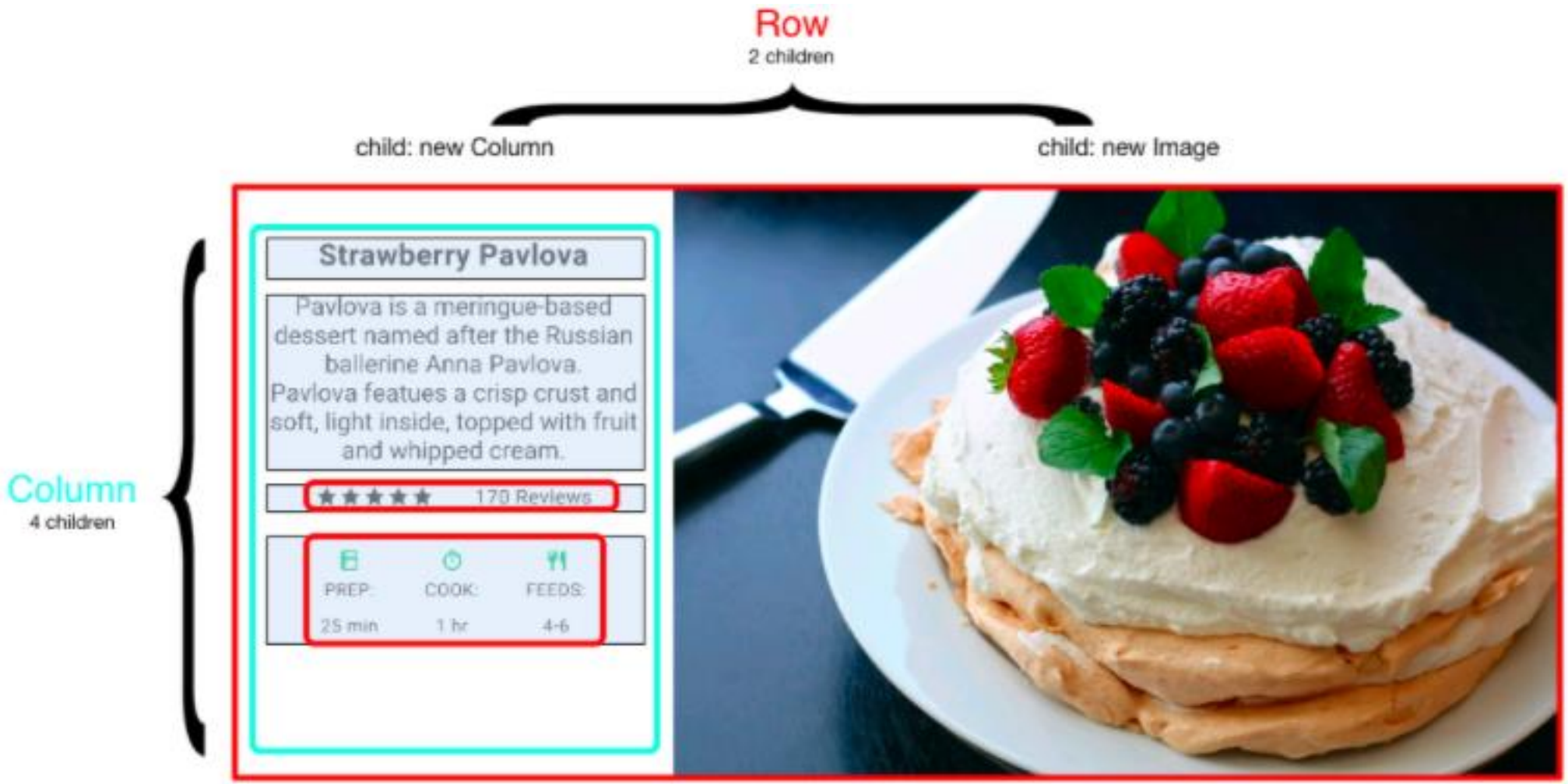
Layout

- O diagrama ao lado representa uma árvore de widgets para montar uma interface de um APP
- É possível construir interfaces com linhas e colunas, umas dentro das outras



Fonte: https://flutter.dev/docs/development/ui/layout

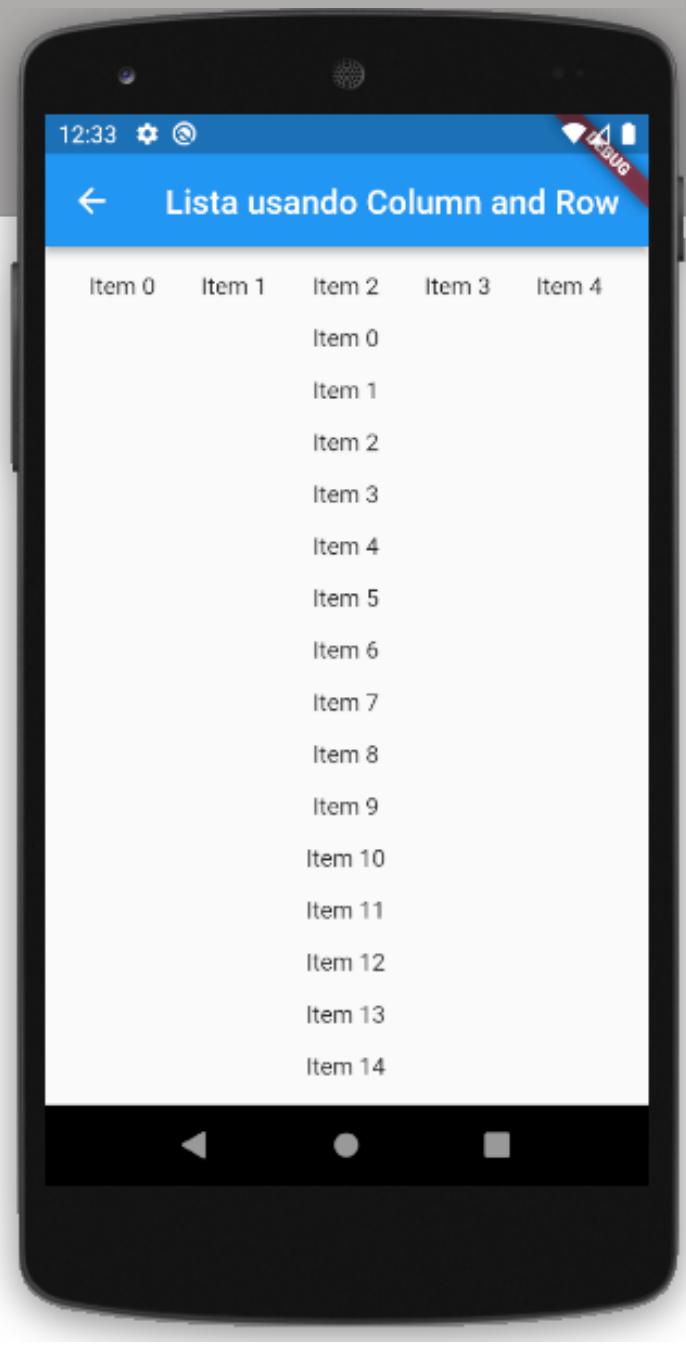
Layout



Fonte: https://flutter.dev/docs/development/ui/layout

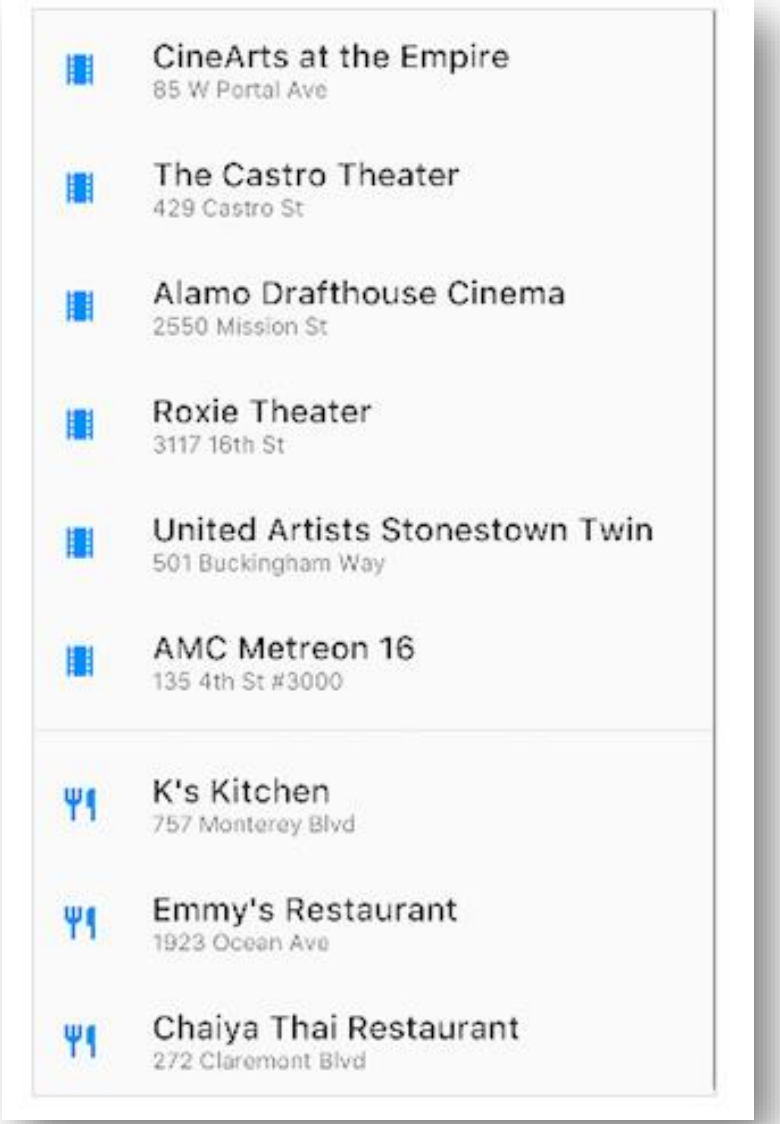
Layout

```
body: Column(
 mainAxisAlignment:
      MainAxisAlignment.spaceEvenly,
      children: [
      Row(
       mainAxisAlignment:
            MainAxisAlignment.spaceEvenly,
       children:
            [
            for(int i=0; i < 5; i++) Text('Item $i'),
            ]),
            for(int i=0; i < 15; i++) Text('Item $i'),
       ])
```



ListView

- ListView é um widget em forma de coluna, que fornece **rolagem automática** quando seu conteúdo é muito longo para sua caixa de renderização
- Pode ser colocado horizontalmente ou verticalmente



Fonte: https://flutter.dev/docs/development/ui/layout#listview

ListView

```
ListView.builder(
  // Let the ListView know how many items it needs to build.
  itemCount: items.length,
  // Provide a builder function. This is where the magic happens.
  // Convert each item into a widget based on the type of item it is.
  itemBuilder: (context, index) {
    final item = items[index];

    return ListTile(
      title: item.buildTitle(context),
      subtitle: item.buildSubtitle(context),
    );
  },
);
```

Para converter cada item em um widget, use o construtor ListView.builder ().

Em geral, forneça uma função de construtor que verifica com qual tipo de item você está lidando e retorne o widget apropriado para esse tipo de item.

Fonte: https://flutter.dev/docs/cookbook/lists/mixed-list

ListView

```
body: Container(
  padding: EdgeInsets.all(20),
  child: ListView.builder(
      itemCount: 5,
      itemBuilder: (context, indice) {
        print("item ${indice}");
        return ListTile(
          title: Text(indice.toString()),
          subtitle: Text("subtitulo"),
        ); // ListTile
      }
      ), // ListView.builder
), // Container
```

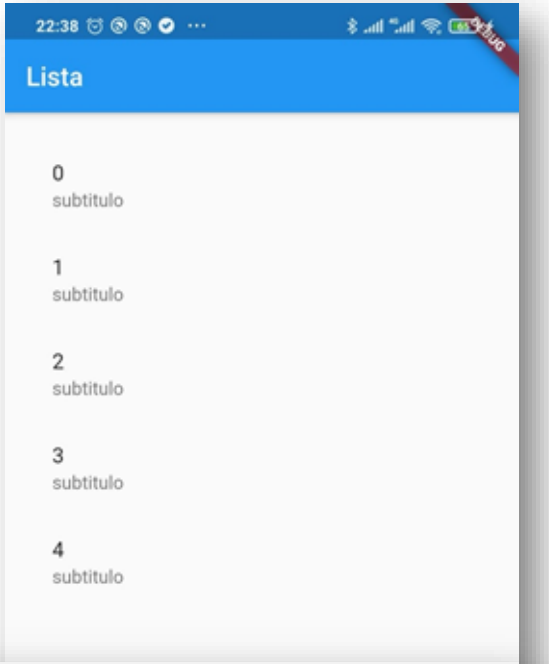
O método builder() usado para converter cada item em um widget

O itemCount será usado para indicar quantas vezes será o builder será executado

Vamos usar um return com o ListTile( ) para escrever na tela um title e um subtitle

ListView

```
body: Container(
  padding: EdgeInsets.all(20),
  child: ListView.builder(
      itemCount: 5,
      itemBuilder: (context, indice) {
        print("item ${indice}");
        return ListTile(
          title: Text(indice.toString()),
          subtitle: Text("subtitulo"),
        ); // ListTile
      }
    ), // ListView.builder
), // Container
```



Console:

```
Console
  oppidx_max 27, oppidx_min 0
I/flutter (16333): item 0
I/flutter (16333): item 1
I/flutter (16333): item 2
I/flutter (16333): item 3
I/flutter (16333): item 4
```

ListView

```
class _HomeState extends State<Home> {

  List _itens = [];

  void _carregarItens(){
    _itens = [];
    for(int i=0; i<=10; i++){
      Map<String, dynamic> item = Map();
      item["titulo"] = "Titulo ${i} da lista";
      item["descricao"]="Descrição ${i} da lista";
      _itens.add(item);
    }
  }
```

Método _carregarItens(), será usado para carregar itens, poderá ser usado para carregar itens da web ou de um banco de dados

```
@override
Widget build(BuildContext context) {
  _carregarItens();
  return Scaffold(
    appBar: AppBar(
      title: Text("Lista"),
    ), // AppBar
    body: Container(...), // Container
  ); // Scaffold
}
}
```

Chamada do método

ListView

```
child: ListView.builder(
    itemCount: _itens.length,
    itemBuilder: (context, indice) {
      Map<String, dynamic> item =_itens[indice];
      print("item ${item["titulo"]}");
      return ListTile(
        title: Text(indice.toString()),
        subtitle: Text("subtitulo"),
      ); // ListTile
    }
  ), // ListView.builder
```

```
child: ListView.builder(
    itemCount: _itens.length,
    itemBuilder: (context, indice) {
      //Map<String, dynamic> item =_itens[indice]
      print("item ${_itens[indice]["titulo"]}");
      return ListTile(
        title: Text(indice.toString()),
        subtitle: Text("subtitulo"),
      ); // ListTile
    }
  ), // ListView.builder
```

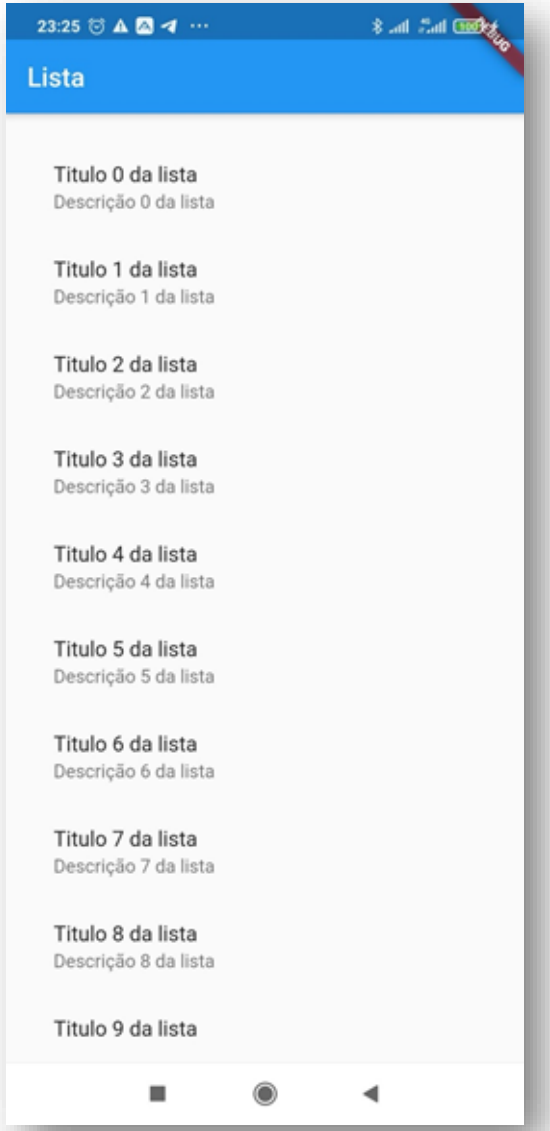
_itens[ ] é uma lista de itens criada no método _carregarItens()

Console

```
I/flutter (24132): item Titulo 0 da lista
I/flutter (24132): item Titulo 1 da lista
I/flutter (24132): item Titulo 2 da lista
I/flutter (24132): item Titulo 3 da lista
I/flutter (24132): item Titulo 4 da lista
I/flutter (24132): item Titulo 5 da lista
```

ListView

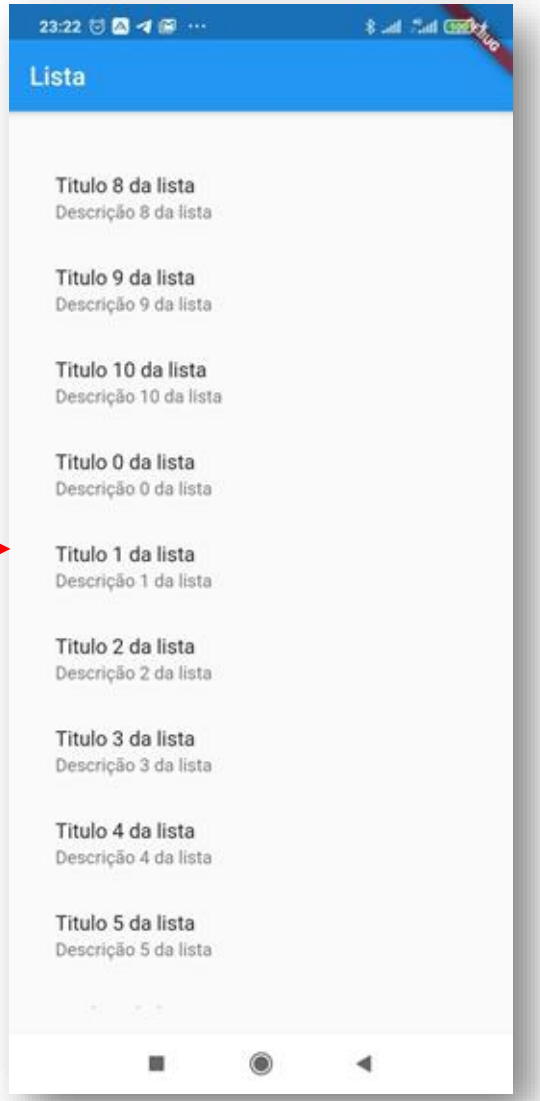
```
body: Container(
  padding: EdgeInsets.all(20),
  child: ListView.builder(
      itemCount: _itens.length,
      itemBuilder: (context, indice) {
        return ListTile(
          title: Text(_itens[indice]["titulo"]),
          subtitle: Text(_itens[indice]["descricao"]),
        ); // ListTile
      }
      ), // ListView.builder
), // Container
```



ListView

```
void _carregarItens(){
  _itens = [];
  for(int i=0; i<=10; i++){
    Map<String, dynamic> item = Map();
    item["titulo"] = "Titulo ${i} da lista";
    item["descricao"]="Descrição ${i} da lista";
    _itens.add(item);
  }
}
```

Para não ficar recarregando sempre a lista novamente ao final como mostrado na tela, vamos “zerar” a lista ao chamar o método _carregarItens()



ListView – onTap e onLongPress

```
return ListTile(
  onTap: (){
    print("Clique com onTap");
  },
  onLongPress: (){
    print("Clique com onLongPress");
  },
  title: Text(_itens[indice]["titulo"]),
  subtitle: Text(_itens[indice]["descricao"]),
); // ListTile
```

onTap: Clique simples no item

onLongPress: Clique e segura o item

ListView – onTap e onLongPress

```
Console
  oppidx_max 27, oppidx_min 0
I/flutter (26072): Clique com onTap
I/flutter (26072): Clique com onLongPress
I/flutter (26072): Clique com onTap
I/flutter (26072): Clique com onLongPress
```

ListView – onTap e onLongPress

```
return ListTile(
  onTap: (){
    print("Clique com onTap ${indice}");
  },
  onLongPress: (){
    print("Clique com onLongPress ${indice}");
  },
  title: Text(_itens[indice]
  subtitle: Text(_itens[indi
); // ListTile
```

Console

```
I/GED      (26072): ged_boost_gpu_freq, level
  oppidx_max 27, oppidx_min 0
I/flutter (26072): Clique com onTap 1
I/flutter (26072): Clique com onLongPress 4
I/flutter (26072): Clique com onTap 6
```

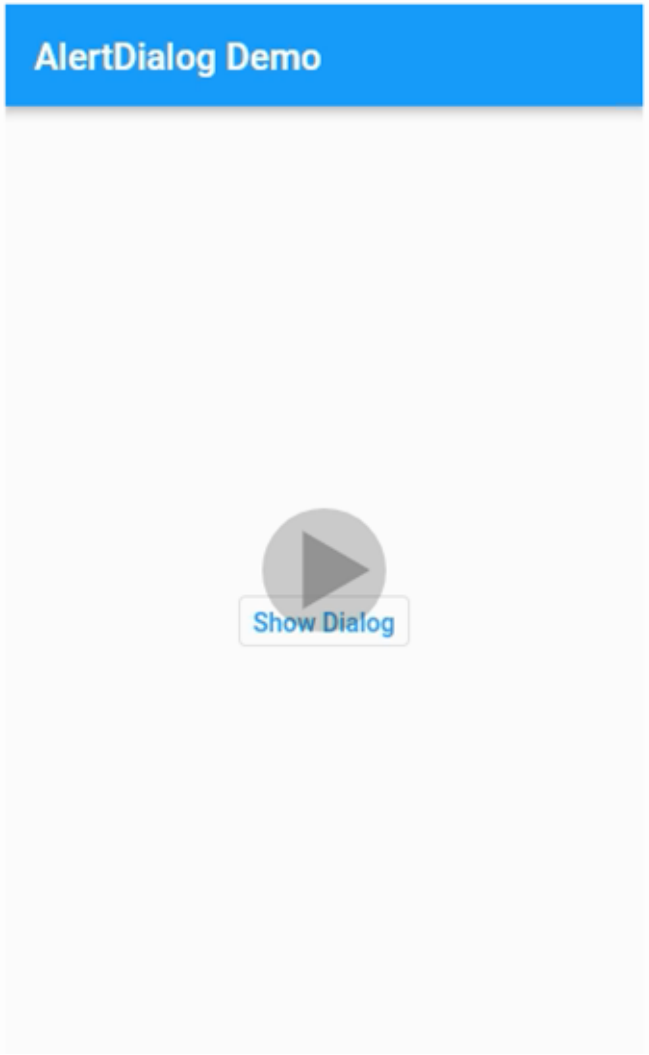
ShowDialog e AlertDialog

- Uma caixa de diálogo de alerta (**AlertDialog**) informa o usuário sobre as situações que requerem reconhecimento
- Um AlertDialog tem um título opcional e uma lista opcional de ações. O título (title) é exibido acima do conteúdo (content) e as ações são exibidas abaixo do conteúdo

```
const AlertDialog({
  Key key,
  this.title,
  this.titlePadding,
  this.titleTextStyle,
  this.content,
  this.contentPadding = const EdgeInsets.fr
  this.contentTextStyle,
  this.actions,
  this.actionsPadding = EdgeInsets.zero,
  this.actionsOverflowDirection,
  this.actionsOverflowButtonSpacing,
  this.buttonPadding,
  this.backgroundColor,
  this.elevation,
  this.semanticLabel,
  this.insetPadding = _defaultInsetPadding,
  this.clipBehavior = Clip.none,
  this.shape,
```

ShowDialog e AlertDialog

```
Future<void> _showMyDialog() async {
  return showDialog<void>(
    context: context,
    barrierDismissible: false, // user must tap button!
    builder: (BuildContext context) {
      return AlertDialog(
        title: Text('AlertDialog Title'),
        content: SingleChildScrollView(
          child: ListBody(
            children: <Widget>[
              Text('This is a demo alert dialog.'),
              Text('Would you like to approve of this message?'),
            ],
          ),
        ),
        actions: <Widget>[
          TextButton(
            child: Text('Approve'),
            onPressed: () {
              Navigator.of(context).pop();
            },
          ),
        ],
```

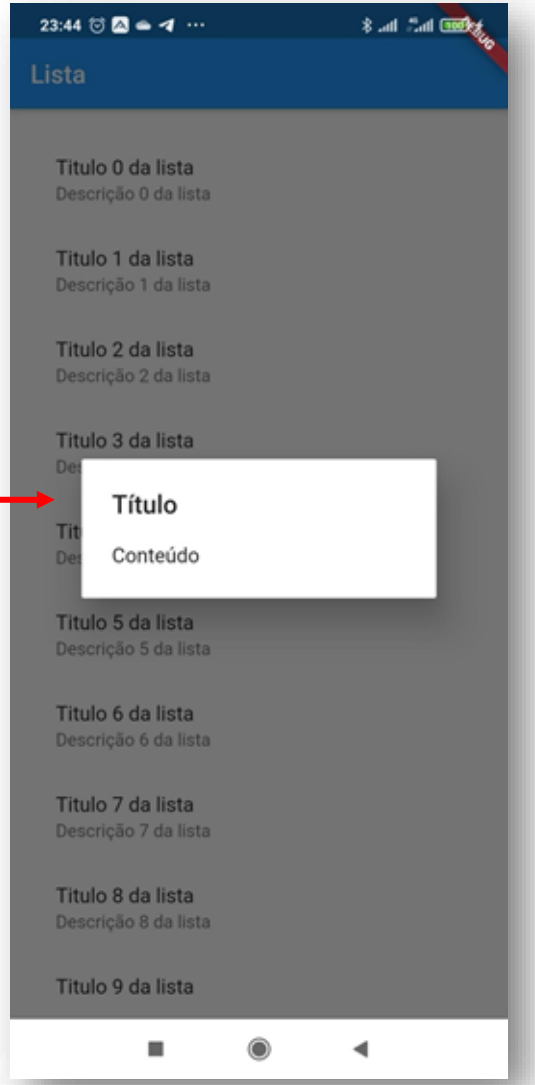


Fonte: https://api.flutter.dev/flutter/material/AlertDialog-class.html

ShowDialog e AlertDialog

```
return ListTile(
  onTap: (){
    //print("Clique com onTap ${indice}");
    showDialog(
        context: context,
        builder: (context){
          return AlertDialog(
            title: Text("Título"),
            content: Text("Conteúdo"),
          ); // AlertDialog
        }
    );
  },
```

Observe que o ShowDialog está dentro do onTap, ou seja, ao clicar em um item da lista



ShowDialog e AlertDialog

```
return ListTile(
  onTap: (){
    //print("Clique com onTap ${indice}");
    showDialog(
        context: context,
        builder: (context){
          return AlertDialog(
            title: Text(_itens[indice]["titulo"]),
            content: Text(_itens[indice]["descricao"]),
          ); // AlertDialog
        }
    );
  },
```

ShowDialog e AlertDialog

```
return ListTile(
  onTap: (){
    //print("Clique com onTap ${indice}");
    showDialog(
        context: context,
        builder: (context){
          return AlertDialog(
            title: Text(_itens[indice]["titulo"]),
            content: Text(_itens[indice]["descricao"]),
          ); // AlertDialog
        }
    );
  },
```

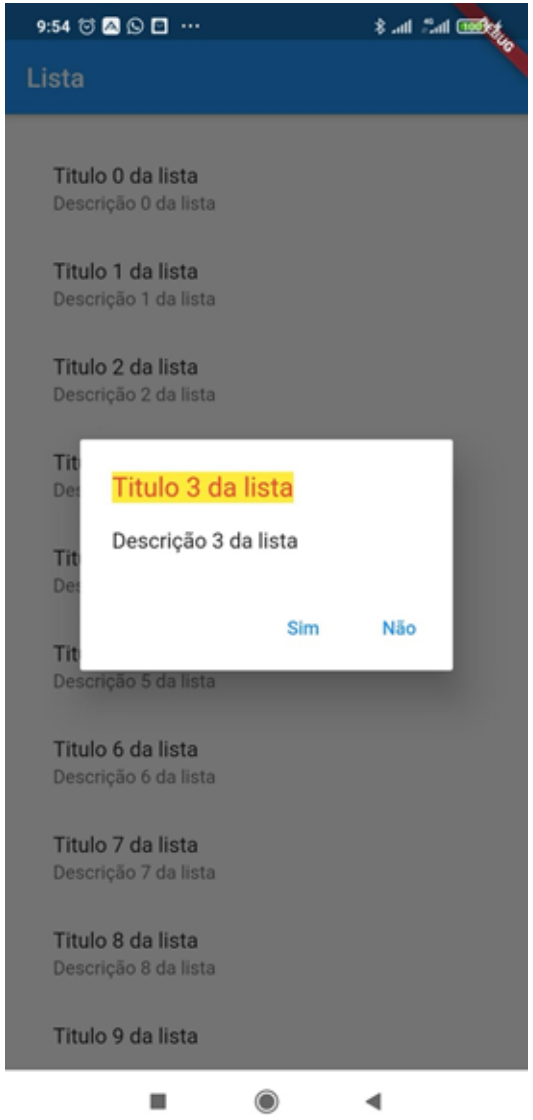
AlertDialog

```
return AlertDialog(
  title: Text(_itens[indice]["titulo"]),
  titleTextStyle: TextStyle(
    fontSize: 20,
    color: Colors.red,
    backgroundColor: Colors.yellow,
  ), // TextStyle
  content: Text(_itens[indice]["descricao"]),
); // AlertDialog
```

AlertDialog

```
return AlertDialog(
  title: Text(_itens[indice]["titulo"]),
  titleTextStyle: TextStyle(...), // TextStyle
  content: Text(_itens[indice]["descricao"]),
  actions: <Widget>[ //definir widgets
    FlatButton(
      onPressed: (){
        print("Selecionado sim");
      },
      child: Text("Sim")
    ), // FlatButton
    FlatButton(
      onPressed: (){
        print("Selecionado não");
      },
      child: Text("Não")
    ), // FlatButton
  ], // <Widget>[]
); // AlertDialog
```

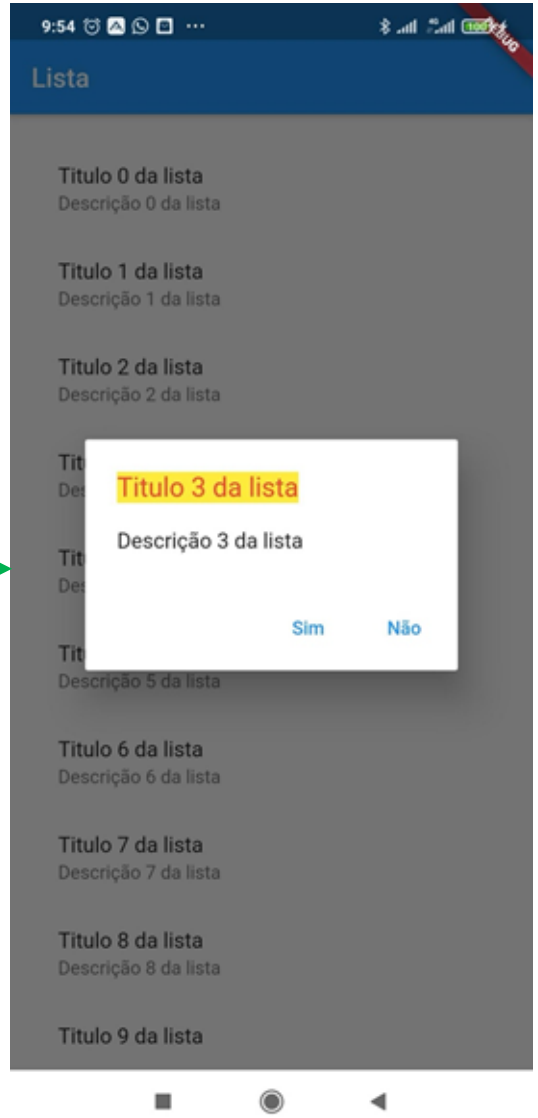
FlatButton foi descontinuado. Use outros botões, como FilledButton, ElevatedButton ou TextButton.



AlertDialog

```
Console
I/flutter (28849): Selecionado não
I/flutter (28849): Selecionado não
I/flutter (28849): Selecionado não
I/flutter (28849): Selecionado sim
I/flutter (28849): Selecionado não
```

Ao clicar no Sim ou Não é impresso no console “Selecionado sim” ou “Selecionado não”. No entanto, não fecha a tela de alerta



AlertDialog

```
actions: <Widget>[ //definir widgets
  FlatButton(
      onPressed: (){
        print("Selecionado sim");
        Navigator.pop(context);
      },
      child: Text("Sim")
  ), // FlatButton
  FlatButton(
      onPressed: (){
        print("Selecionado não");
        Navigator.pop(context);
      },
      child: Text("Não")
  ), // FlatButton
], // <Widget>[]
```

Use Navigator.pop() para fechar a tela do AlertDialog ao clicar no Sim ou no Não

Prática - Entregar

1. Crie uma página com uma lista de itens conforme o estudo dirigido desta aula. Sua lista deve ter pelo menos 20 itens. Adicione uma AlertDialog para responder ao clique em cada item da lista com o título “Alerta” e a descrição “Você clicou no item [número do item]”, em que [número do item] corresponde ao índice do item na lista. O usuário deve ser capaz de fechar a AlertDialog ao clicar em “sim” ou “não”.

(continua no próximo slide.)

Prática - Entregar

2. Após, modifique a tela de login criada nas aulas passadas ou crie uma nova, conforme descrito a seguir.
   a. Na tela de login, o usuário deve informar nome, e-mail e senha. A aplicação deve avaliar se o e-mail e senha digitados são "admin@admin.com" e "12345", respectivamente.
   b. Caso não seja, deve-se mostrar uma AlertDialog com título "Dados inválidos" e descrição "Usuário e/ou senha incorreto(a)". Caso contrário, deve-se ir para a tela criada no exercício 1 acima a partir de uma rota nomeada.
   c. Modifique a tela do exercício 1 para receber uma string nome e exibir no título de seu appBar "Bem vindo(a), ${nome}". A tela de login deve fornecer a string.

Referências Bibliográficas

- Curso da Udemy – Flutter Essencial do professor Ricardo Lecheta.
- Curso da Udemy - Desenvolvimento Android e IOS com Flutter 2020 – Crie 15 Apps do professor Jamilton Damasceno.
- Site Flutter – flutter.dev
  - https://flutter.dev/docs/development/ui/layout
  - https://flutter.dev/docs/cookbook/lists/mixed-list
  - https://flutter.dev/docs/development/ui/layout#listview
  - https://api.flutter.dev/flutter/material/AlertDialog-class.html